value of *one*. When this is multiplied by the weights of the hidden layer, it provides a *bias* (DC offset) to each sigmoid. This addition is called a **bias node**. It is treated the same as the other nodes, except for the constant input.

Can neural networks be made without a sigmoid or similar nonlinearity? To answer this, look at the three-layer network of Fig. 26-5. If the sigmoids were not present, the three layers would *collapse* into only two layers. In other words, the summations and weights of the hidden and output layers could be combined into a single layer, resulting in only a two-layer network.

Why Does It Work?

The weights required to make a neural network carry out a particular task are found by a **learning algorithm**, together with **examples** of how the system *should* operate. For instance, the examples in the sonar problem would be a database of several hundred (or more) of the 1000 sample segments. Some of the example segments would correspond to submarines, others to whales, others to random noise, etc. The learning algorithm uses these examples to calculate a set of weights appropriate for the task at hand. The term **learning** is widely used in the neural network field to describe this process; however, a better description might be: *determining an optimized set of weights based on the statistics of the examples*. Regardless of what the method is called, the resulting weights are virtually impossible for humans to understand. Patterns may be observable in some rare cases, but generally they appear to be random numbers. A neural network using these weights can be observed to have the proper input/output relationship, but *why* these particular weights work is quite baffling. This mystic quality of neural networks has caused many scientists and engineers to shy away from them. Remember all those science fiction movies of renegade computers taking over the earth?

In spite of this, it is common to hear neural network advocates make statements such as: "neural networks are well understood." To explore this claim, we will first show that it is possible to pick neural network weights through traditional DSP methods. Next, we will demonstrate that the learning algorithms provide *better* solutions than the traditional techniques. While this doesn't explain *why* a particular set of weights works, it does provide confidence in the method.

In the most sophisticated view, the neural network is a method of labeling the various regions in *parameter space*. For example, consider the sonar system neural network with 1000 inputs and a single output. With proper weight selection, the output will be near *one* if the input signal is an echo from a submarine, and near *zero* if the input is only noise. This forms a parameter hyperspace of 1000 dimensions. The neural network is a method of assigning a value to each location in this hyperspace. That is, the 1000 input values define a *location* in the hyperspace, while the output of the neural network provides the *value* at that location. A look-up table could perform this task perfectly, having an output value stored for each possible input address. The